

Robotics 2

Dynamic model of robots: Newton-Euler approach

Prof. Alessandro De Luca

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI



SAPIENZA
UNIVERSITÀ DI ROMA

Approaches to dynamic modeling
(reprise)



energy-based approach (Euler-Lagrange = EL)

- multi-body robot seen as a whole
- constraint (internal) reaction forces between the links are automatically eliminated: in fact, they do not perform work
- closed-form (symbolic) equations are directly obtained
- best suited for study of dynamic properties and **analysis** of control schemes

Newton-Euler (NE) method (balance of forces/moments)

- dynamic equations written separately for each link/body
- mainly used for inverse dynamics in real time
  - equations are evaluated in a numeric and recursive way
  - best for **synthesis** (=implementation) of model-based control schemes
- by eliminating the internal reaction forces and performing back-substitution of all expressions, we get dynamic equations in closed-form (identical to Euler-Lagrange!)

Derivative of a vector in a moving frame



... from velocity to acceleration

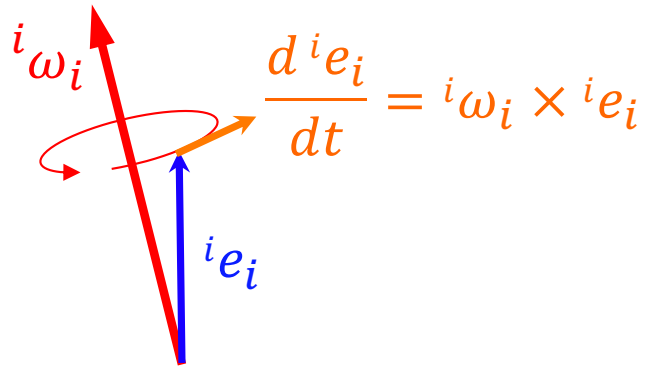
$$^0v_i = {}^0R_i\ {}^iv_i \qquad\qquad {}^0\dot{R}_i = S({}^0\omega_i)\ {}^0R_i$$

$$^0\dot{v}_i = {}^0a_i = {}^0R_i\ {}^ia_i = {}^0R_i\ {}^i\dot{v}_i + {}^0\dot{R}_i\ {}^iv_i$$

$$= {}^0R_i\ {}^i\dot{v}_i + {}^0\omega_i \times {}^0R_i\ {}^iv_i = {}^0R_i\ ({}^i\dot{v}_i + {}^i\omega_i \times {}^iv_i)$$

$$^ia_i = {}^i\dot{v}_i + {}^i\omega_i \times {}^iv_i$$

derivative of a "unit" vector in a moving frame



Dynamics of a rigid body

- Newton dynamic equation
  - balance: sum of forces = variation of linear momentum

$$\sum f_i = \frac{d}{dt}(mv_c) = m\dot{v}_c$$

- Euler dynamic equation
  - balance: sum of moments = variation of angular momentum

$$\sum \mu_i = \frac{d}{dt}(I\omega) = I\dot{\omega} + \frac{d}{dt}(R\bar{I}R^T)\,\omega = I\dot{\omega} + \left(\dot{R}\bar{I}R^T + R\bar{I}\dot{R}^T\right)\omega$$

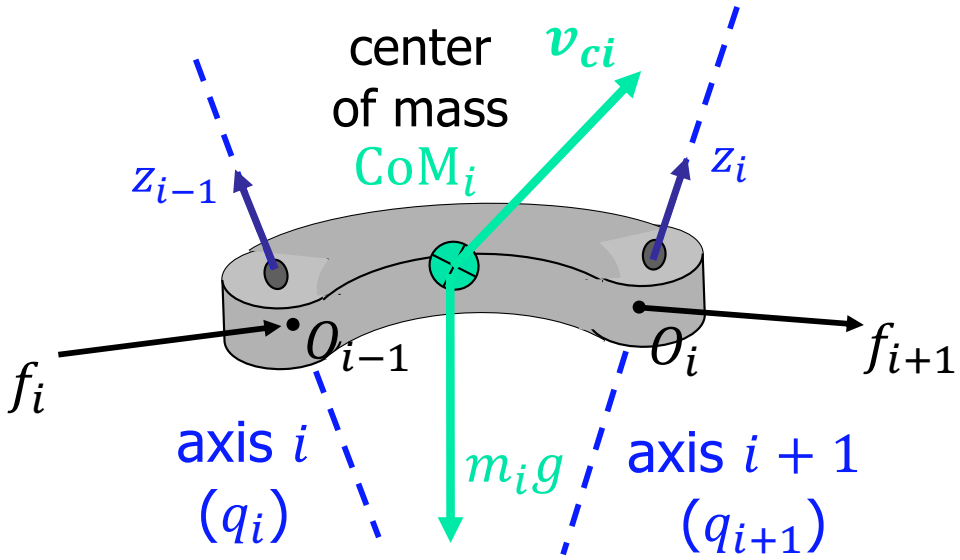
$$= I\dot{\omega} + S(\omega)R\bar{I}R^T\omega + R\bar{I}R^TS^T(\omega)\omega = I\dot{\omega} + \omega \times I\omega$$

- principle of action and reaction
  - forces/moments: applied by body i to body $i+1$

    $= -$ applied by body $i+1$ to body i

Newton-Euler equations - 1

link i



FORCES

f_i force applied from link $i-1$ on link i

f_{i+1} force applied from link i on link $i+1$

$m_i g$ gravity force

all vectors expressed in the same RF (better in RF_i ...)

Newton equation

$$f_i - f_{i+1} + m_i g = m_i a_{ci}$$

N

linear acceleration of CoM_i

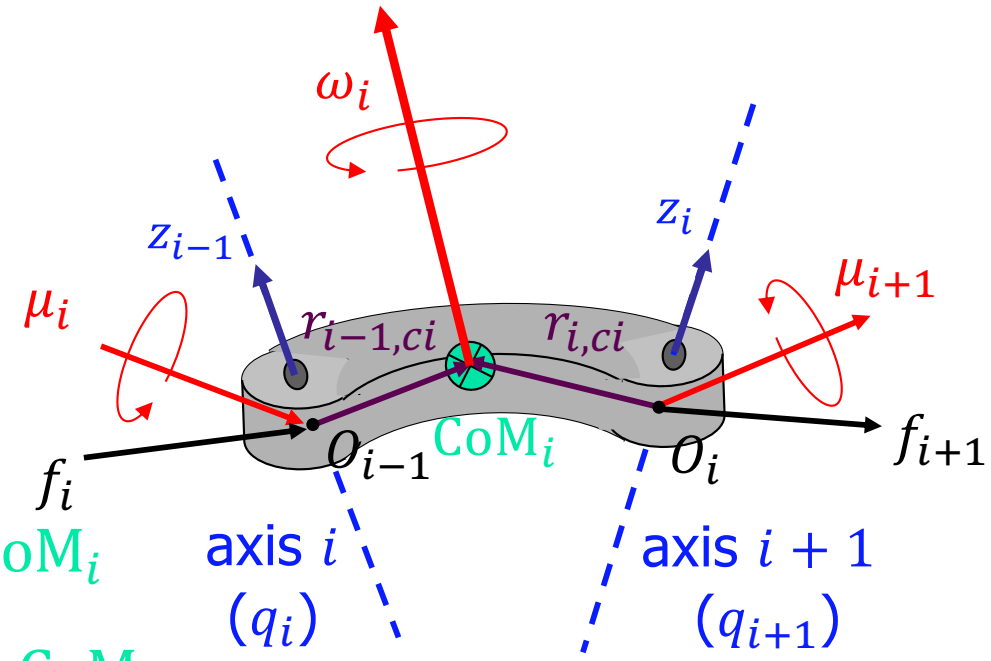
Newton-Euler equations - 2

link i

MOMENTS

μ_i moment applied
from link $(i-1)$ on link i

μ_{i+1} moment applied
from link i on link $(i+1)$



$f_i \times r_{i-1,ci}$ moment due to f_i w.r.t. CoM_i

$-f_{i+1} \times r_{i,ci}$ moment due to $-f_{i+1}$ w.r.t. CoM_i

gravity force gives no moment at CoM_i

all vectors expressed in the same RF (... RF_i !!)

Euler equation

$$\mu_i - \mu_{i+1} + f_i \times r_{i-1,ci} - f_{i+1} \times r_{i,ci} = I_i \dot{\omega}_i + \omega_i \times (I_i \omega_i)$$

E

angular acceleration of body i

Forward recursion
Computing velocities and accelerations



- "moving frames" algorithm (as for velocities in EL)
- for simplicity, only revolute joints here
  (see textbook for the more general treatment)

initializations

$z_0 = (0\ 0\ 1)^T$

$$^i\omega_i = {}^{i-1}R_i^T\left[{}^{i-1}\omega_{i-1} + \dot{q}_i\,{}^{i-1}z_{i-1}\right] \quad \leftarrow {}^0\omega_0$$

$$^i\dot{\omega}_i = {}^{i-1}R_i^T\left[{}^{i-1}\dot{\omega}_{i-1} + \ddot{q}_i\,{}^{i-1}z_{i-1}\right] + {}^{i-1}\dot{R}_i^T\left[{}^{i-1}\omega_{i-1} + \dot{q}_i\,{}^{i-1}z_{i-1}\right]$$

AVR $$= {}^{i-1}R_i^T\left[{}^{i-1}\dot{\omega}_{i-1} + \ddot{q}_i\,{}^{i-1}z_{i-1} + \dot{q}_i\,{}^{i-1}\omega_{i-1}\times{}^{i-1}z_{i-1}\right] \quad \leftarrow {}^0\dot{\omega}_0$$

$$^i a_i = {}^{i-1}R_i^T\,{}^{i-1}a_{i-1} + {}^i\dot{\omega}_i \times {}^i r_{i-1,i} + {}^i\omega_i \times \left({}^i\omega_i \times {}^i r_{i-1,i}\right) \quad \leftarrow {}^0a_0 - {}^0g$$

$$^i a_{ci} = {}^i a_i + {}^i\dot{\omega}_i \times {}^i r_{i,ci} + {}^i\omega_i \times \left({}^i\omega_i \times {}^i r_{i,ci}\right)$$

the gravity force term can be skipped in Newton equation, **if added here**

Proof of recursion
for the angular acceleration ${}^{i}\dot{\omega}_i$



$$
{}^{i}\omega_i = {}^{i-1}R_i^T\left[{}^{i-1}\omega_i\right] = {}^{i-1}R_i^T\left[{}^{i-1}\omega_{i-1} + {}^{i-1}\omega_{i-1,i}\right] = {}^{i-1}R_i^T\left[{}^{i-1}\omega_{i-1} + \dot{q}_i\,{}^{i-1}z_{i-1}\right]
$$

contribution due to revolute joint i

same formula used in recursive algorithm for kinetic energy

$$
\begin{aligned}
{}^{i}\dot{\omega}_i &= {}^{i-1}R_i^T\left[{}^{i-1}\dot{\omega}_{i-1} + \ddot{q}_i\,{}^{i-1}z_{i-1}\right] + {}^{i-1}\dot{R}_i^T\left[{}^{i-1}\omega_i\right] \\
&= {}^{i-1}R_i^T\left[{}^{i-1}\dot{\omega}_{i-1} + \ddot{q}_i\,{}^{i-1}z_{i-1}\right] + \left(S\left({}^{i-1}\omega_{i-1,i}\right){}^{i-1}R_i\right)^T\left[{}^{i-1}\omega_i\right] \\
&= {}^{i-1}R_i^T\left[{}^{i-1}\dot{\omega}_{i-1} + \ddot{q}_i\,{}^{i-1}z_{i-1}\right] - {}^{i-1}R_i^T S\left({}^{i-1}\omega_{i-1,i}\right)\left[{}^{i-1}\omega_i\right] \qquad (S^T = -S) \\
&= \quad \ldots \quad - {}^{i-1}R_i^T\left[{}^{i-1}\omega_{i-1,i} \times {}^{i-1}\omega_i\right] \\
&= \quad \ldots \quad - {}^{i-1}R_i^T\left[\dot{q}_i\,{}^{i-1}z_{i-1} \times \left({}^{i-1}\omega_{i-1} + \dot{q}_i\,{}^{i-1}z_{i-1}\right)\right] \\
&= \quad \ldots \quad - {}^{i-1}R_i^T\left[\dot{q}_i\,{}^{i-1}z_{i-1} \times {}^{i-1}\omega_{i-1}\right] = \quad \ldots \quad + {}^{i-1}R_i^T\left[\dot{q}_i\,{}^{i-1}\omega_{i-1} \times {}^{i-1}z_{i-1}\right] \\
&= {}^{i-1}R_i^T\left[{}^{i-1}\dot{\omega}_{i-1} + \ddot{q}_i\,{}^{i-1}z_{i-1} + \dot{q}_i\,{}^{i-1}\omega_{i-1} \times {}^{i-1}z_{i-1}\right]
\end{aligned}
$$

Hint: by direct time differentiation of the general DH rotation matrix ${}^{i-1}R_i(\theta_i)$ of a revolute joint show that

$$
{}^{i-1}\dot{R}_i = S\left({}^{i-1}\omega_{i-1,i}\right){}^{i-1}R_i \qquad \text{with } {}^{i-1}\omega_{i-1,i} = \dot{\theta}_i \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \dot{\theta}_i\,{}^{i-1}z_{i-1}
$$

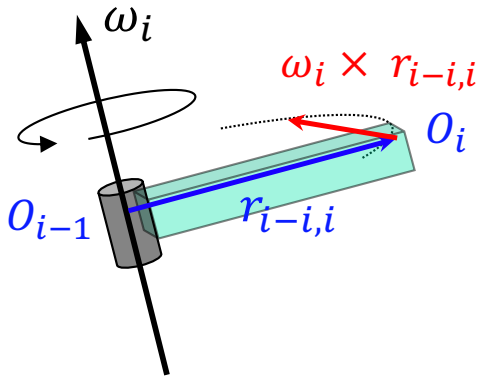
Proof of recursion
for the acceleration ${}^{i}a_i$

in any fixed frame

$$v_i = v_{i-1} + \omega_i \times r_{i-i,i}$$

fundamental kinematic relation of a rigid body
(relation between velocities of point O_{i-1} and O_i)

$$\dot{v}_i = \dot{v}_{i-1} + \dot{\omega}_i \times r_{i-i,i} + \omega_i \times \dot{r}_{i-i,i}$$
$$= \dot{v}_{i-1} + \dot{\omega}_i \times r_{i-i,i} + \omega_i \times (\omega_i \times r_{i-i,i})$$



in moving frames

$${}^{i}a_i = {}^{i}\dot{v}_i = {}^{i-1}R_i^T\,{}^{i-1}\dot{v}_{i-1} + {}^{i}\dot{\omega}_i \times {}^{i}r_{i-1,i} + {}^{i}\omega_i \times ({}^{i}\omega_i \times {}^{i}r_{i-1,i})$$
$$= {}^{i-1}R_i^T\,{}^{i-1}a_{i-1} + {}^{i}\dot{\omega}_i \times \underbrace{{}^{i}r_{i-1,i}}_{\text{constant for a revolute joint}} + {}^{i}\omega_i \times ({}^{i}\omega_i \times \underbrace{{}^{i}r_{i-1,i}}_{\text{constant for a revolute joint}})$$

Backward recursion
Computing forces and moments

from N_i ⟶ to N_{i-1}

eliminated, if inserted in forward recursion (i=0)

initializations

F/MR

$$^{i}f_i = {}^{i}R_{i+1}\,{}^{i+1}f_{i+1} + m_i\,({}^{i}a_{ci} - {}^{i}g) \quad \longleftarrow \quad f_{n+1}$$

$$^{i}\mu_i = {}^{i}R_{i+1}\,{}^{i+1}\mu_{i+1} + ({}^{i}R_{i+1}\,{}^{i+1}f_{i+1}) \times {}^{i}r_{i,ci} - {}^{i}f_i \times ({}^{i}r_{i-1,i} + {}^{i}r_{i,ci}) + {}^{i}I_i\,{}^{i}\dot{\omega}_i + {}^{i}\omega_i \times {}^{i}I_i\,{}^{i}\omega_i \quad \longleftarrow \quad \mu_{n+1}$$

from E_i ⟶ to E_{i-1}

at each recursion step, the two vector equations ($N_i + E_i$) at joint i provide a wrench $(f_i,\ \mu_i) \in \mathbb{R}^6$): this contains also reaction forces/moments at the joint axis ⇒ to be "projected" along/around this axis to produce work

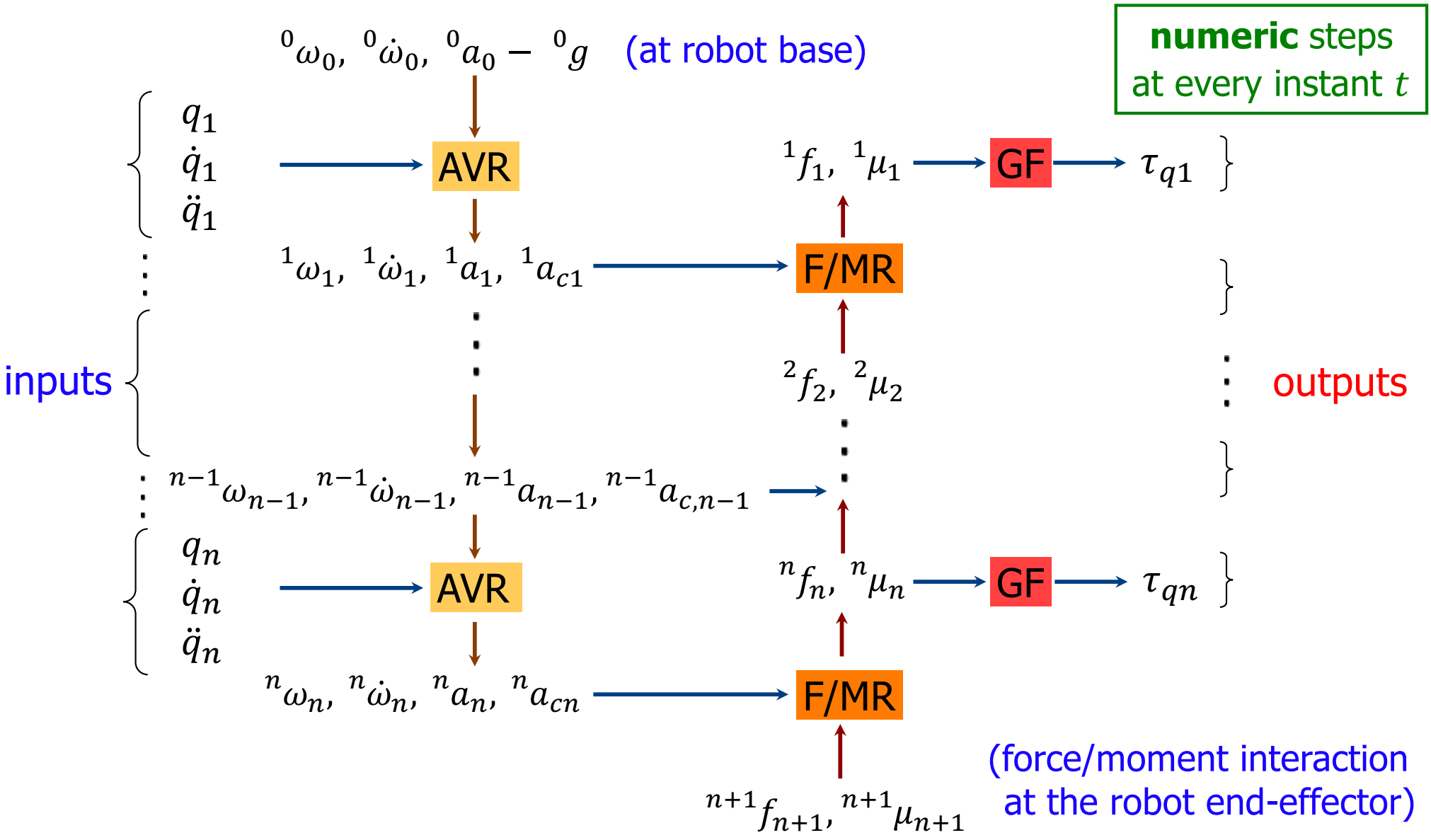
GF $$\tau_{qi} = \begin{cases} {}^{i}f_i^T\ {}^{i}z_{i-1} & \text{for a prismatic joint} \\ {}^{i}\mu_i^T\ {}^{i}z_{i-1} & \text{for a revolute joint} \end{cases}$$

generalized force

➡ n scalar equations at the end

Newton-Euler algorithm

efficient computational scheme for inverse dynamics



Adding extra dynamic terms



- torque $\tau_q \in \mathbb{R}^n$ is associated to a robot link motion $(q, \dot{q}, \ddot{q})$ at time t
- to obtain such motion by delivering a torque $\tau \in \mathbb{R}^n$ with the motors

$$\tau_{qi} = \tau_i \qquad i = 1, \ldots, n$$

- to include other dynamic terms in the NE recursion
  - dissipative terms at the joints (e.g., viscous and Coulomb) …

$$\tau_{qi} = \tau_i + B_{vi}\,\dot{q}_i + B_{ci}\,\mathrm{sign}(\dot{q}_i)$$

  - … and (reflected) motor inertias

$$\tau_{qi} + I_{mi} n_{ri}^2\,\ddot{q}_i = \underbrace{\tau_i + B_{vi}\,\dot{q}_i + B_{ci}\,\mathrm{sign}(\dot{q}_i)}_{\text{nonconservative terms}}$$

- considering an interaction with the environment at the robot end-effector

$$F = (f_{n+1}, \mu_{n+1}) \neq 0 \quad \Rightarrow \quad \left.\begin{matrix} {}^i f_i^T & {}^i z_{i-1} \\ {}^i \mu_i^T & {}^i z_{i-1} \end{matrix}\right\} = \tau_{qi} + \tau_{\mathrm{env},i}$$

inizialization of F/MR

$$\tau_{\mathrm{int}} = -\,\tau_{\mathrm{env}}$$
$$\tau_{\mathrm{int}} = J^T(q) F$$

Comments on Newton-Euler method



- the previous forward/backward recursive formulas can be evaluated in symbolic or numeric form
  - symbolic
    - substituting expressions in a recursive way
    - at the end, a closed-form dynamic model is obtained, which is identical to the one obtained using EL method (or any other method)
    - there is no special convenience in using NE in this way …
  - numeric
    - substituting numeric values (numbers!) at each step
    - computational complexity of each step remains constant ⇒ grows in a linear fashion with the number n of joints ($O(n)$)
    - strongly recommended for real-time use, especially when the number n of joints is large

Script with NE algorithm
(MATLAB, Python, C, …)



general routine $u = NE_{\alpha}(\arg_1, \arg_2, \arg_3)$

assume no interaction with environment ($f_{n+1} = \mu_{n+1} = 0$)

- data file (of a specific robot)
  - number n and types $\sigma = \{0,1\}^n$ of joints (revolute/prismatic)
  - table of DH kinematic parameters
  - list of ALL dynamic parameters of the links (and of the motors)
- input
  - vector parameter $\alpha = \{{}^0g, 0\}$ (presence or absence of gravity)
  - three ordered vector arguments
    - typically, samples of joint position, velocity, acceleration taken from a desired trajectory
- output
  - generalized force u for the complete inverse dynamics
  - … or single terms of the dynamic model

Examples of output

- complete inverse dynamics

$$u = NE^{0}{}_{g}(q_d, \dot{q}_d, \ddot{q}_d) = M(q_d)\ddot{q}_d + c(q_d, \dot{q}_d) + g(q_d) = \tau_d$$

- gravity term

$$u = NE^{0}{}_{g}(q, 0, 0) = g(q)$$

- centrifugal and Coriolis term

$$u = NE_0(q, \dot{q}, 0) = c(q, \dot{q})$$

- i-th column of the inertia matrix

$$u = NE_0(q, 0, e_i) = M_i(q)$$

e_i = i-th column of identity matrix

- generalized momentum

$$u = NE_0(q, 0, \dot{q}) = M(q)\dot{q} = p$$

- … environment interaction at end-effector level

$$F = (f_{n+1}, \mu_{n+1}) \neq 0 \quad \Rightarrow \quad u = NE_0(q, 0, 0) = \tau_{\text{int}}$$

A further example of output

- factorization of centrifugal and Coriolis term

$$u = NE_0(q, \dot{q}, 0) = c(q, \dot{q}) = C(q, \dot{q})\dot{q}$$

$C(q, \dot{q})$ with Christoffel symbols

- for later use, what about a "mixed" velocity term?

$$C(q, \dot{q})\dot{q}_r \;\Leftrightarrow\; \begin{cases} u = NE_0(q, \dot{q}_r, 0) = C(q, \dot{q}_r)\dot{q}_r \\ u = NE_0(q, e_i\dot{q}_{ri}, 0) = C_i(q, e_i\dot{q}_{ri})\dot{q}_{ri} \end{cases}$$

no good!

a) $C(q, \dot{q})\dot{q}_r = C(q, \dot{q}_r)\dot{q}$, when using Christoffel symbols

b) $C(q, \dot{q} + \dot{q}_r)(\dot{q} + \dot{q}_r) = C(q, \dot{q})\dot{q} + C(q, \dot{q}_r)\dot{q}_r + 2C(q, \dot{q})\dot{q}_r$

$$\Rightarrow \; u = \frac{1}{2}\left(NE_0(q, \dot{q} + \dot{q}_r, 0) - NE_0(q, \dot{q}, 0) - NE_0(q, \dot{q}_r, 0)\right)$$
$$= C(q, \dot{q})\dot{q}_r$$

(i.e., with 3 calls of standard NE algorithm)

[Kawasaki et al., IEEE T-RA 1996]

Modified NE algorithm



modified routine $\widehat{NE}_\alpha(\arg_1, \arg_2, \arg_3, \arg_4)$ with 4 arguments

[De Luca, Ferrajoli, ICRA 2009]

$$\widehat{NE}_\alpha(x, y, y, z) = NE_\alpha(x, y, z)$$ consistency property

e.g., $u = \widehat{NE}_{g}(q, 0, 0, 0) = NE_{g}(q, 0, 0) = g(q)$

$$u = \widehat{NE}_0(q, \dot{q}, \dot{q}, 0) = NE_0(q, \dot{q}, 0) = c(q, \dot{q}) = C(q, \dot{q})\dot{q}$$

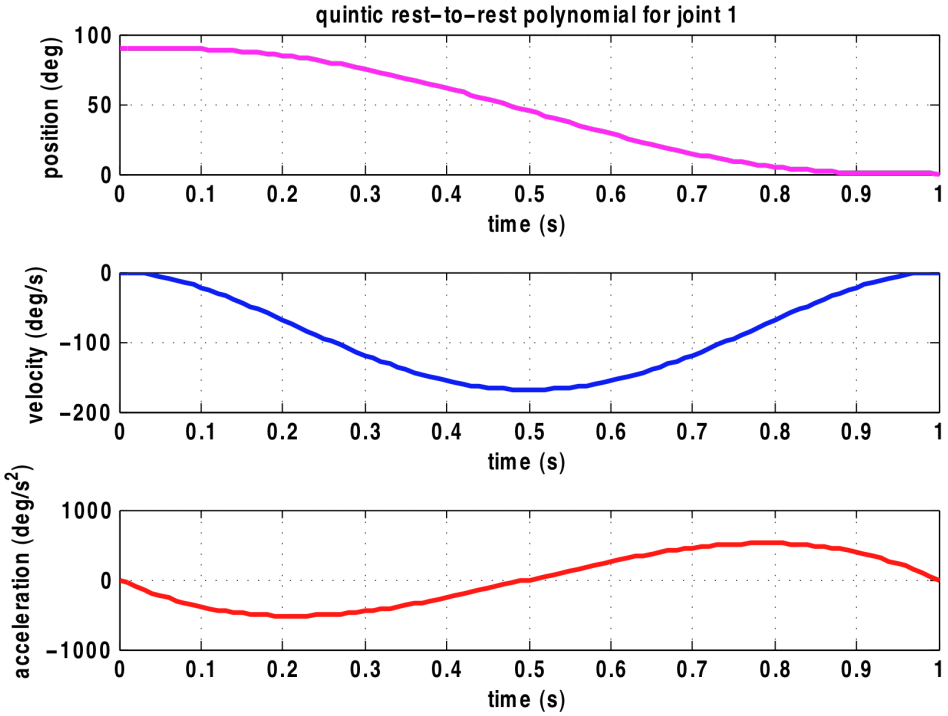
$\Rightarrow$ $u = \widehat{NE}_0(q, \dot{q}, \dot{q}_r, 0) = C(q, \dot{q})\dot{q}_r$ with $\dot{M} - 2C$ skew-symmetric

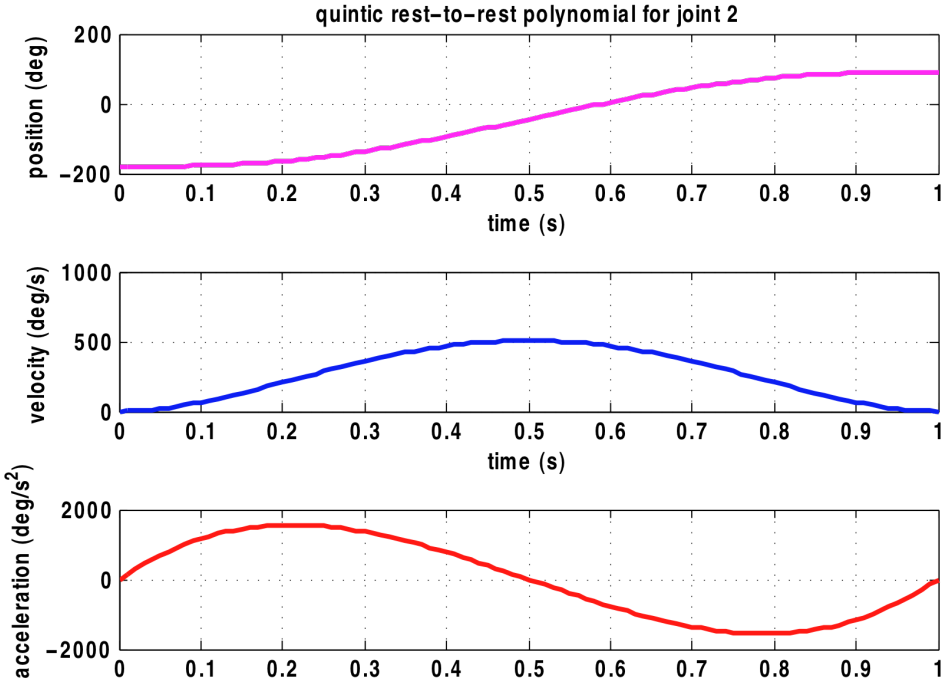
(i.e., with 1 call of modified NE algorithm)

$\Rightarrow$ $u = \widehat{NE}_0(q, \dot{q}, e_i, 0) = C_i(q, \dot{q})$

(i.e., the full matrix C satisfying the skew-symmetry of $\dot{M} - 2C$ with n calls of the modified NE algorithm)

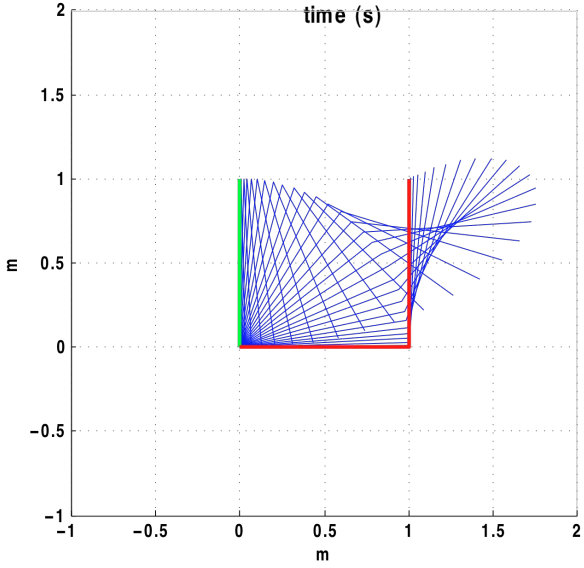
Inverse dynamics of a 2R planar robot





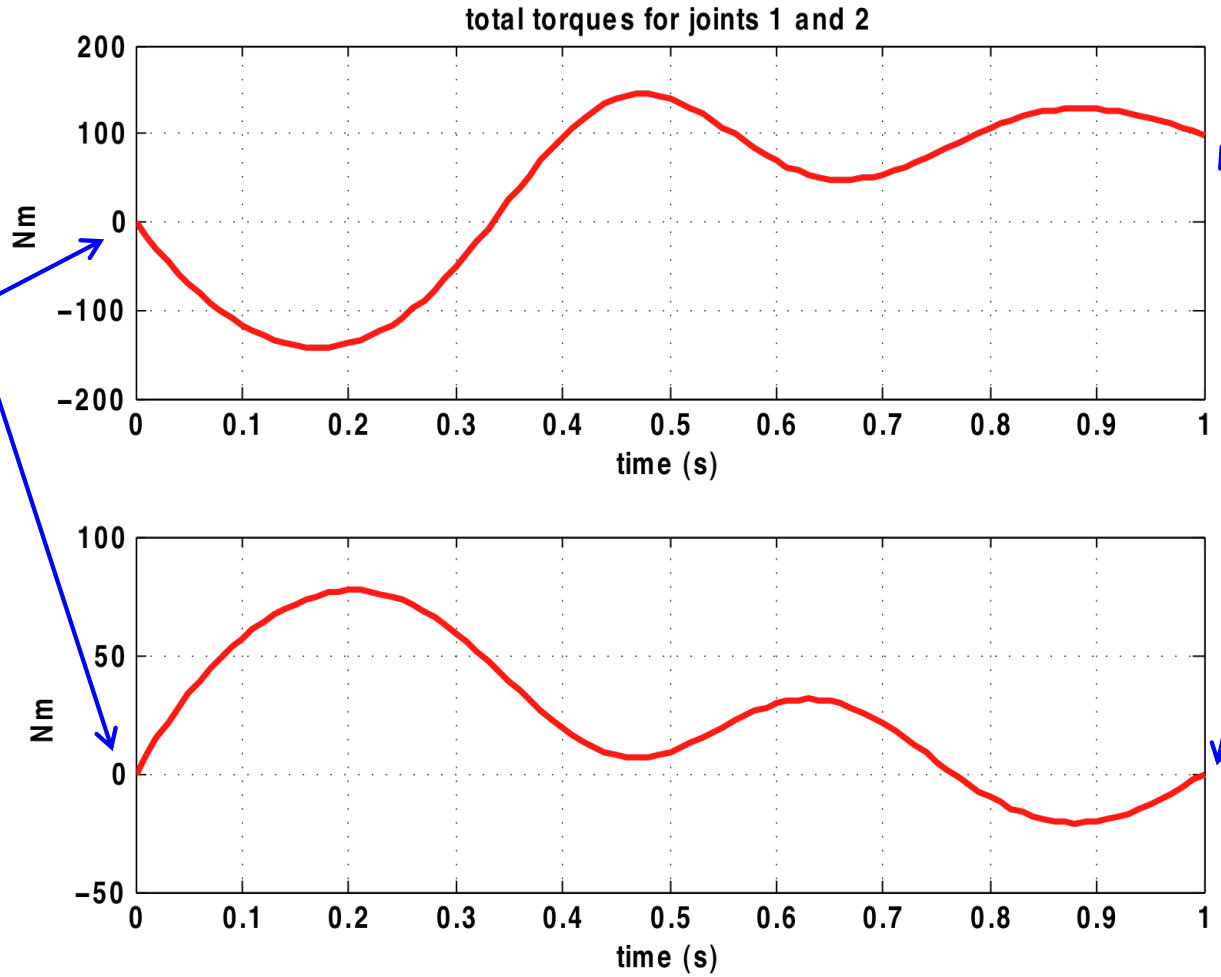
desired (smooth) joint motion:
quintic polynomials for q_1, q_2 with
zero vel/acc boundary conditions
from (90°, -180°) to (0°, 90°) in $T = 1$ s

$\Leftrightarrow$



Inverse dynamics of a 2R planar robot

zero initial torques = free equilibrium configuration + zero initial accelerations

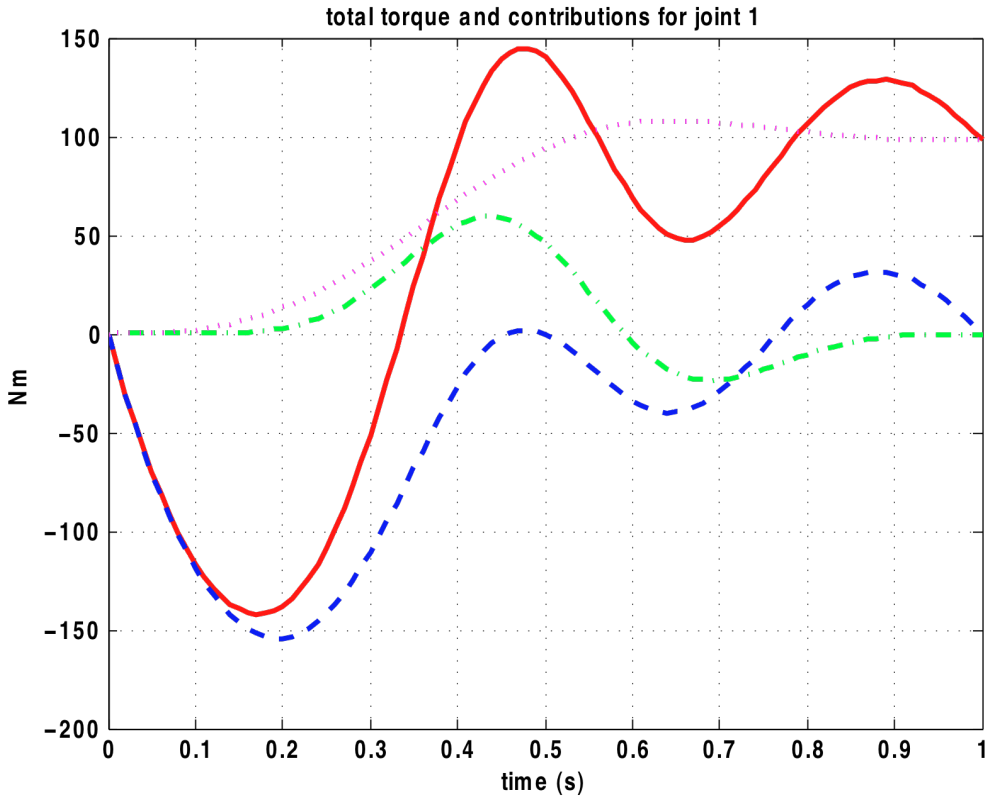


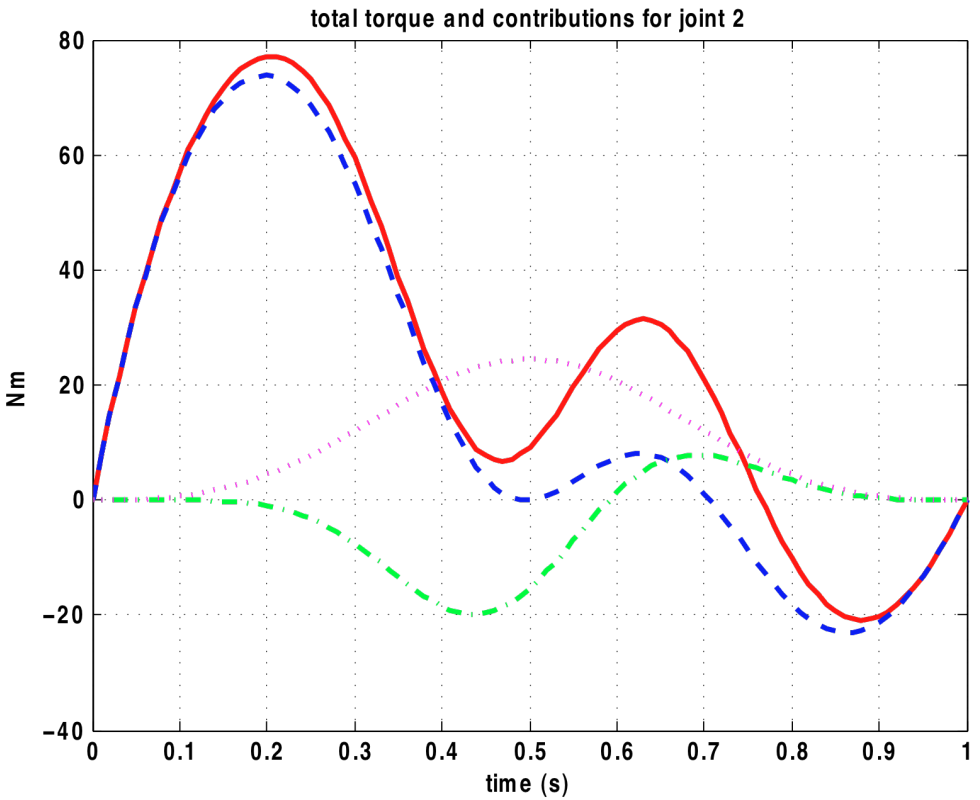
final torques $u_1 \neq 0, u_2 = 0$ balance link weights in final (0°, 90°) configuration

motion in vertical plane (under gravity)

both links are thin rods of uniform mass $m_1 = 10$ kg, $m_2 = 5$ kg

Inverse dynamics of a 2R planar robot





torque contributions at the two joints for the desired motion

——— = total, - - - - = inertial

-·-·- = Coriolis/centrifugal, ····· = gravitational

Use of NE routine for simulation
direct dynamics



- numerical integration, at current state $(q, \dot{q})$, of

$$\ddot{q} = M^{-1}(q)[u - (c(q,\dot{q}) + g(q))] = M^{-1}(q)[u - \tilde{n}(q,\dot{q})]$$

- Coriolis, centrifugal, and gravity terms

$$\tilde{n} = NE^{0}{}_{g}(q,\dot{q},0) \qquad \text{complexity } O(n)$$

- i-th column of the inertia matrix, for $i = 1,..,n$

$$M_i = NE_0(q,0,e_i) \qquad O(n^2)$$

- numerical inversion of inertia matrix

$$InvM = \text{inv}(M) \qquad O(n^3)$$

though with small coefficient

- given u, integrate acceleration computed as

$$\ddot{q} = InvM * [u - \tilde{n}] \;\Rightarrow$$

integrate for new state $(q, \dot{q})$ and repeat over time ...